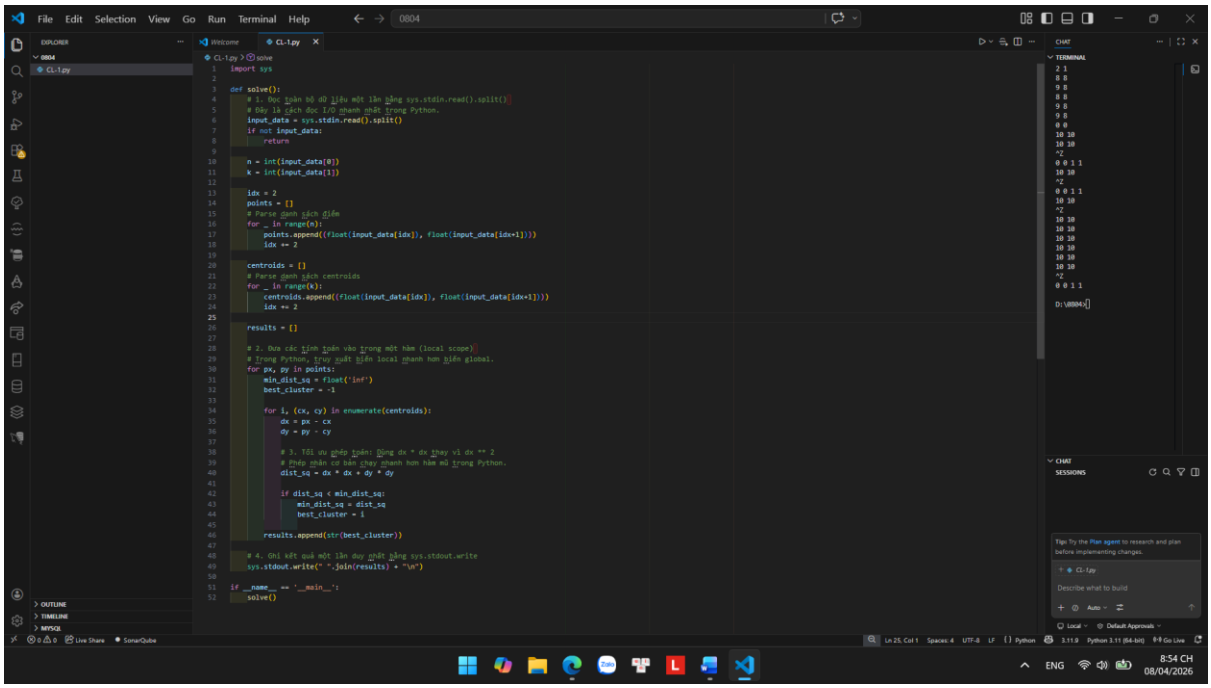
CL-1



```python
import sys

def solve():
    # 1. Đọc toàn bộ dữ liệu một lần bằng sys.stdin.read().split()
    # Đây là cách đọc I/O nhanh nhất trong Python.
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    k = int(input_data[1])

    idx = 2
    points = []
    # Parse danh sách điểm
    for _ in range(n):
        points.append((float(input_data[idx]), float(input_data[idx+1])))
        idx += 2

    centroids = []
    # Parse danh sách centroids
    for _ in range(k):
        centroids.append((float(input_data[idx]), float(input_data[idx+1])))
        idx += 2

    results = []

    # 2. Đưa các tính toán vào trong một hàm (local scope)
    # Trong Python, truy xuất biến local nhanh hơn biến global.
    for px, py in points:
        min_dist_sq = float('inf')
        best_cluster = -1

        for i, (cx, cy) in enumerate(centroids):
            dx = px - cx
            dy = py - cy

            # 3. Tối ưu phép toán: Dùng dx * dx thay vì dx ** 2
            # Phép nhân cơ bản chạy nhanh hơn hàm mũ trong Python.
            dist_sq = dx * dx + dy * dy

            if dist_sq < min_dist_sq:
                min_dist_sq = dist_sq
                best_cluster = i

        results.append(str(best_cluster))

    # 4. Ghi kết quả một lần duy nhất bằng sys.stdout.write
    sys.stdout.write(" ".join(results) + "\n")

if __name__ == '__main__':
    solve()
```

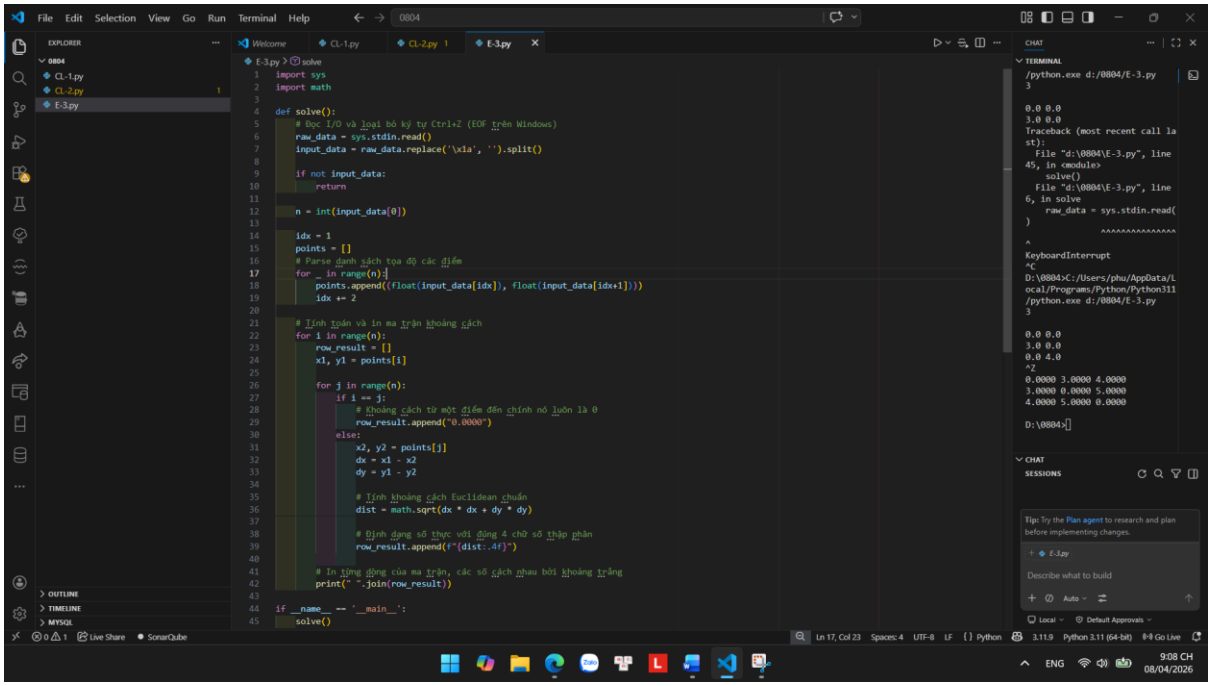
CL-2

```python
import sys

def solve():
    # Đọc I/O và loại bỏ ký tự Ctrl+Z (EOF trên Windows) nếu có
    raw_data = sys.stdin.read()
    input_data = raw_data.replace('\x1a', '').split()

    if not input_data:
        return

    n = int(input_data[0])
    k = int(input_data[1])
    t = int(input_data[2])

    idx = 3
    points = []
    # Parse n điểm
    for _ in range(n):
        points.append((float(input_data[idx]), float(input_data[idx+1])))
        idx += 2

    centroids = []
    # Parse k tâm (dùng list thay vì tuple để có thể cập nhật lại tọa độ)
    for _ in range(k):
        centroids.append([float(input_data[idx]), float(input_data[idx+1])])
        idx += 2

    labels = [-1] * n

    # Thực hiện đúng t vòng lặp K-Means
    for _ in range(t):

        # BƯỚC 1: Gán mỗi điểm vào tâm gần nhất
        for p_idx in range(n):
            px, py = points[p_idx]
            min_dist_sq = float('inf')
            best_cluster = -1
```

```
10 10
^Z
0 0 1 1

D:\0804>C:/Users/phu/AppData
/Local/Programs/Python/Pytho
n311/python.exe d:/0804/CL-2
.py
4 2 2
1 1
2 1
8 8
9 8
0 0
10 10
^Z
0 0 1 1
1.50 1.00
8.50 8.00

D:\0804>
```

```python
def solve():
            dy = py - cy
            dist_sq = dx * dx + dy * dy

            if dist_sq < min_dist_sq:
                min_dist_sq = dist_sq
                best_cluster = c_idx

        labels[p_idx] = best_cluster

    # BƯỚC 2: Cập nhật lại tọa độ các tâm (là trung bình cộng của các điểm trong cụm)
    sum_x = [0.0] * k
    sum_y = [0.0] * k
    counts = [0] * k

    for p_idx in range(n):
        cluster = labels[p_idx]
        sum_x[cluster] += points[p_idx][0]
        sum_y[cluster] += points[p_idx][1]
        counts[cluster] += 1

    for c_idx in range(k):
        if counts[c_idx] > 0:
            centroids[c_idx][0] = sum_x[c_idx] / counts[c_idx]
            centroids[c_idx][1] = sum_y[c_idx] / counts[c_idx]
        # Lưu ý: Nếu counts[c_idx] == 0 (cụm rỗng), giữ nguyên tâm cũ theo luật của đề bài.

    # IN KẾT QUẢ ĐẦU RA
    # Dòng 1: Nhãn của các điểm
    print(" ".join(map(str, labels)))

    # k dòng tiếp theo: Tọa độ tâm cuối cùng, làm tròn 2 chữ số thập phân
    for cx, cy in centroids:
        print(f"{cx:.2f} {cy:.2f}")

if __name__ == '__main__':
    solve()
```


```
10 10
^Z
0 0 1 1

D:\0804>C:/Users/phu/AppData
/Local/Programs/Python/Pytho
n311/python.exe d:/0804/CL-2
.py
4 2 2
1 1
2 1
8 8
9 8
0 0
10 10
^Z
0 0 1 1
1.50 1.00
8.50 8.00

D:\0804>
```

VIRTUAL PROGRAMMING LAB

Lab-DataClustering-3